

Application of R for Finance - Individual Assignment

Aleya Zaki

2025-11-23

Table of contents

Acknowledgement	4
Part 1: Data Loading & Cleaning	4
Part 2: Compute Daily Returns & Autocorrelation Function	5
Part 2.1: ACF of Daily Returns	6
Part 2.2: Weekly Mean Returns	7
Part 2.3: ACF of Weekly Mean Returns	8
Part 3: Subset data to 2020 onwards	9
Part 4: Generate Moving Average crossover signals for Pfizer	10
Part 5: Backtest this strategy & Plot Performance	11
Part 5: Optimisation loop for fast/slow windows	12
Part 5.2: Line plot for the best MA crossover strategy	14
Part 5.3: Performance Summary for the Best Strategy	16
Part 6: Bollinger Bands Breakout Strategy	17
Concluding Analysis	19

List of Figures

1	Autocorrelation of Pfizer Daily Returns	7
2	Autocorrelation of Pfizer Weekly Mean Returns	9
3	Moving Average Crossover Strategy	12
4	Line Plot Pfizer Price with Moving Averages	16
5	Best Performing Strategy of Moving Average Crossover Strategy	17
6	Bollinger Bands Breakout Strategy	19

List of Tables

1	First six observations of cleaned Pfizer (PFE) daily data	5
2	First six observations of PFE daily return	6
3	First six observations of PFE weekly return	8
4	First six observations of PFE daily return from 2020 onwards	10

5	Best-performing Moving Average Window Combination Based on Cumulative Strategy Returns	14
---	---	----

```
library(dplyr)
library(dtplyr)
library(data.table)
library(lubridate)
library(ggplot2)
library(PerformanceAnalytics)
library(xts)
library(zoo)

source("strategy.R")
```

Acknowledgement

I acknowledge the use of ChatGPT-5 (OpenAI, <https://chatgpt.com>) in completing this assignment. AI was used to draft a version of the Bollinger Bands Breakout function and to help interpret coding errors during developments. All final code, parameter choices, strategy implementation, and interpretation of results represent my own work and understanding. Therefore, AI was used solely as a support tool to aid learning and debugging, not to replace independent analysis.

Part 1: Data Loading & Cleaning

In this section, I load the large Compustat daily dataset using `fread()`, then dropped columns I will not use, followed by subsetting it to Pfizer (PFE) and selecting only columns relevant for later analysis.

```
big_data <- fread("compustat_daily_2010_2025.csv")

# Load big data efficiently and drop obviously unused columns
big_data <- fread(
  "compustat_daily_2010_2025.csv",
  drop = c("exchg", "prchd", "prcld")
)

# Filter for Pfizer (ticker PFE)
pfe_data <- big_data %>%
  filter(tic == "PFE")

# Convert the date column
```

```

pfe_data <- pfe_data %>%
  mutate(
    datadate = mdy(datadate)
  ) %>%
  arrange(datadate)

# Keep only the columns we actually need for returns + strategies
pfe <- pfe_data %>%
  select(datadate, tic, prccd)

# Show first few rows as required in the assignment
head(pfe)

```

A data.table: 6×3

Table 1: First six observations of cleaned Pfizer (PFE) daily data

datadate <date>	tic <chr>	prccd <dbl>
2010-01-04	PFE	18.93
2010-01-05	PFE	18.66
2010-01-06	PFE	18.60
2010-01-07	PFE	18.53
2010-01-08	PFE	18.68
2010-01-11	PFE	18.83

Part 2: Compute Daily Returns & Autocorrelation Function

In this step, I compute simple daily returns for Pfizer and plot the autocorrelation function (ACF) to examine whether daily returns exhibit time dependence.

```

pfe_returns <- pfe %>%
  arrange(datadate) %>%
  mutate(
    daily_return = (prccd - lag(prccd)) / lag(prccd)
  ) %>%
  filter(!is.na(daily_return))
head(pfe_returns)

```

A data.table: 6×4

Table 2: First six observations of PFE daily return

datadate <date>	tic <chr>	prccd <dbl>	daily_return <dbl>
2010-01-05	PFE	18.66	-0.014263074
2010-01-06	PFE	18.60	-0.003215434
2010-01-07	PFE	18.53	-0.003763441
2010-01-08	PFE	18.68	0.008094981
2010-01-11	PFE	18.83	0.008029979
2010-01-12	PFE	18.77	-0.003186405

Part 2.1: ACF of Daily Returns

```
acf(
  pfe_returns$daily_return,
  main = "Autocorrelation of Pfizer Daily Returns"
)
```

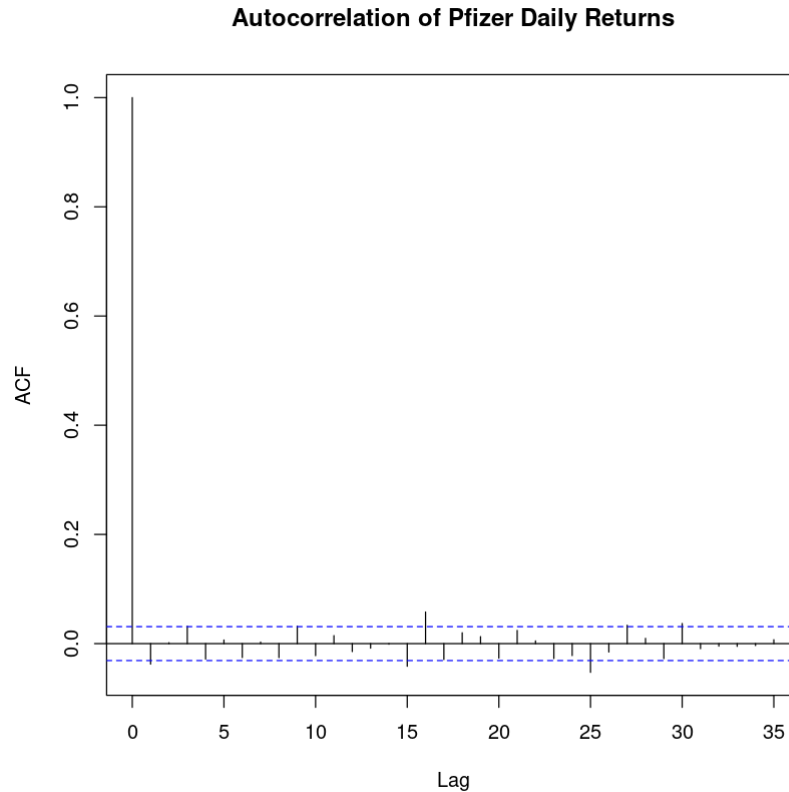


Figure 1: Autocorrelation of Pfizer Daily Returns

Part 2.2: Weekly Mean Returns

To obtain deeper insights, I aggregate returns to weekly frequency and compute autocorrelation of the weekly mean returns. This allows detection of medium-term dependence that may not appear in daily data.

```
pfe_weekly <- pfe_returns %>%  
  mutate(week = floor_date(datadate, "week")) %>%  
  group_by(week) %>%  
  summarise(weekly_return = mean(daily_return, na.rm = TRUE)) %>%  
  ungroup()  
head(pfe_weekly)
```

A tibble: 6 × 2

Table 3: First six observations of PFE weekly return

week <date>	weekly_return <dbl>
2010-01-03	-0.003286742
2010-01-10	0.008562150
2010-01-17	-0.006622766
2010-01-24	-0.003132801
2010-01-31	-0.007375525
2010-02-07	-0.001771689

Part 2.3: ACF of Weekly Mean Returns

```
acf(  
  pfe_weekly$weekly_return,  
  main = "Autocorrelation of Pfizer Weekly Mean Returns"  
)
```

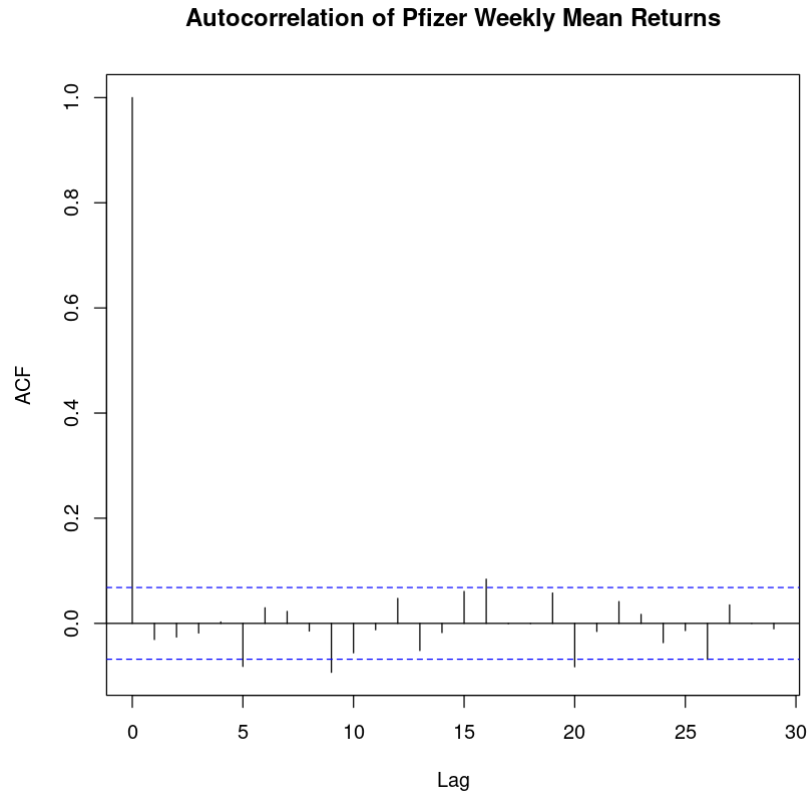



Figure 2: Autocorrelation of Pfizer Weekly Mean Returns

Part 3: Subset data to 2020 onwards

Here, I subset Pfizer's returns data to include only observations from 2020-01-01 onwards. All subsequent trading strategy analysis, including moving average crossover and Bollinger Bands, will be based on this 2020+ dataset.

```
pfe_2020 <- pfe_returns %>%  
  filter(datadate >= as.Date("2020-01-01")) %>%  
  select(datadate, tic, prccd, daily_return)  
head(pfe_2020)
```

A data.table: 6 × 4

Table 4: First six observations of PFE daily return from 2020 onwards

datadate <date>	tic <chr>	prccd <dbl>	daily_return <dbl>
2020-01-02	PFE	39.14	-0.001020929
2020-01-03	PFE	38.93	-0.005365355
2020-01-06	PFE	38.88	-0.001284357
2020-01-07	PFE	38.75	-0.003343621
2020-01-08	PFE	39.06	0.008000000
2020-01-09	PFE	38.89	-0.004352279

```
# Ensure the data is sorted by date
pfe_2020 <- pfe_2020 %>% arrange(datadate)

# Convert close prices to xts
pfe_price_xts <- xts(pfe_2020$prccd, order.by = pfe_2020$datadate)
colnames(pfe_price_xts) <- "Close"

# Convert daily returns to xts
pfe_return_xts <- xts(pfe_2020$daily_return, order.by = pfe_2020$datadate)
colnames(pfe_return_xts) <- "daily_return"

head(pfe_price_xts)
```

```
      Close
2020-01-02 39.14
2020-01-03 38.93
2020-01-06 38.88
2020-01-07 38.75
2020-01-08 39.06
2020-01-09 38.89
```

Part 4: Generate Moving Average crossover signals for Pfizer

```
# Generate the MA strategy
fast_w <- 20
slow_w <- 60

pfe_ma_xts <- ma_signal_roll_xts(
  price_xts = pfe_price_xts,
```

```

    fast_window = fast_w,
    slow_window = slow_w
)

```

```

head(pfe_ma_xts)
tail(pfe_ma_xts)

```

	fast_MA	slow_MA	signal
2020-01-02	NA	NA	NA
2020-01-03	NA	NA	NA
2020-01-06	NA	NA	NA
2020-01-07	NA	NA	NA
2020-01-08	NA	NA	NA
2020-01-09	NA	NA	NA

	fast_MA	slow_MA	signal
2025-10-22	25.1695	24.76050	1
2025-10-23	25.2230	24.77483	1
2025-10-24	25.2730	24.79933	1
2025-10-27	25.3190	24.82067	1
2025-10-28	25.2700	24.83683	1
2025-10-29	25.1240	24.82917	1

Part 5: Backtest this strategy & Plot Performance

```

# Backtest using lecturer function (it computes returns itself)
ma_results <- backtest(
  xts_prices = pfe_price_xts,
  xts_signals = pfe_ma_xts[, "signal"]
)

# Plot performance summary
charts.PerformanceSummary(
  ma_results,
  main = "Pfizer: Market vs Strategy (MA Crossover)"
)

```

Pfizer: Market vs Strategy (MA Crossover)



Figure 3: Moving Average Crossover Strategy

Part 5: Optimisation loop for fast/slow windows

In this section, I loop over different fast/slow moving-average windows, backtest each combination, and select the one with the highest cumulative strategy return.

```
fast_grid <- c(20, 30, 40, 50)
slow_grid <- c(60, 90, 120, 150)

results <- data.frame(
  fast = integer(),
  slow = integer(),
  cum_return = numeric()
)

for (f in fast_grid) {
```

```

for (s in slow_grid) {

  if (f >= s) next # skip invalid combos

  # 1) Generate xts signals
  tmp_ma_xts <- ma_signal_roll_xts(
    price_xts = pfe_price_xts,
    fast_window = f,
    slow_window = s
  )

  # 2. Backtest (no plot)
  tmp_returns_xts <- backtest(
    xts_prices = pfe_price_xts,
    xts_signals = tmp_ma_xts[, "signal"]
  )

  # 3. Cumulative return of the Strategy
  strat_cum_ret <- Return.cumulative(
    tmp_returns_xts[, "Strategy"],
    geometric = TRUE
  )

  results <- rbind(
    results,
    data.frame(
      fast = f,
      slow = s,
      cum_return = as.numeric(strat_cum_ret)
    )
  )
}
}

# Choose the best combo
best_row <- results[which.max(results$cum_return), ]

best_row <- as.data.frame(best_row)

best_row

```

A data.frame: 1 × 3

Table 5: Best-performing Moving Average Window Combination Based on Cumulative Strategy Returns

	fast <dbl>	slow <dbl>	cum_return <dbl>
15	50	120	0.1787666

Part 5.2: Line plot for the best MA crossover strategy

Next, I visualise the best-performing moving average crossover strategy by plotting the Pfizer price series together with the fast and slow moving averages. Vertical lines indicate where crossovers (and thus signal changes) occur.

```
best_fast <- best_row$fast
best_slow <- best_row$slow

# 1. Recompute signals for the best combo
pfe_best <- ma_signal_roll_xts(
  price_xts      = pfe_price_xts,
  fast           = best_fast,
  slow           = best_slow
)

# 2. Convert xts to a data frame from ggplot
pfe_best_df <- data.frame(
  datadate = index(pfe_best),
  price     = as.numeric(pfe_price_xts[index(pfe_best)]),
  fast_MA   = as.numeric(pfe_best[, "fast_MA"]),
  slow_MA   = as.numeric(pfe_best[, "slow_MA"]),
  signal     = as.numeric(pfe_best[, "signal"])
)

# 3. Identify crossover
pfe_best_df <- pfe_best_df %>%
  arrange(datadate) %>%
  mutate(signal_change = signal != lag(signal))

crossovers <- pfe_best_df %>%
  filter(signal_change == TRUE & !is.na(signal_change))
```

```

# 4. Plot with colours + crossover highlights
ggplot(pfe_best_df, aes(x = datadate)) +
  geom_line(aes(y = price, colour = "Price"), linewidth = 0.4) +
  geom_line(aes(y = fast_MA, colour = "Fast MA"), linewidth = 0.4) +
  geom_line(aes(y = slow_MA, colour = "Slow MA"), linewidth = 0.4) +
  geom_vline(
    data = crossovers,
    aes(xintercept = datadate),
    linetype = "dashed",
    alpha = 0.4
  ) +
  scale_colour_manual(
    name = "Legend",
    values = c(
      "Price" = "black",
      "Fast MA" = "forestgreen",
      "Slow MA" = "red"
    )
  ) +
  labs(
    title = paste0(
      "Pfizer Price with Moving Averages (fast = ",
      best_fast, ", slow = ", best_slow, ")"
    ),
    x = "Date",
    y = "Price"
  )

```

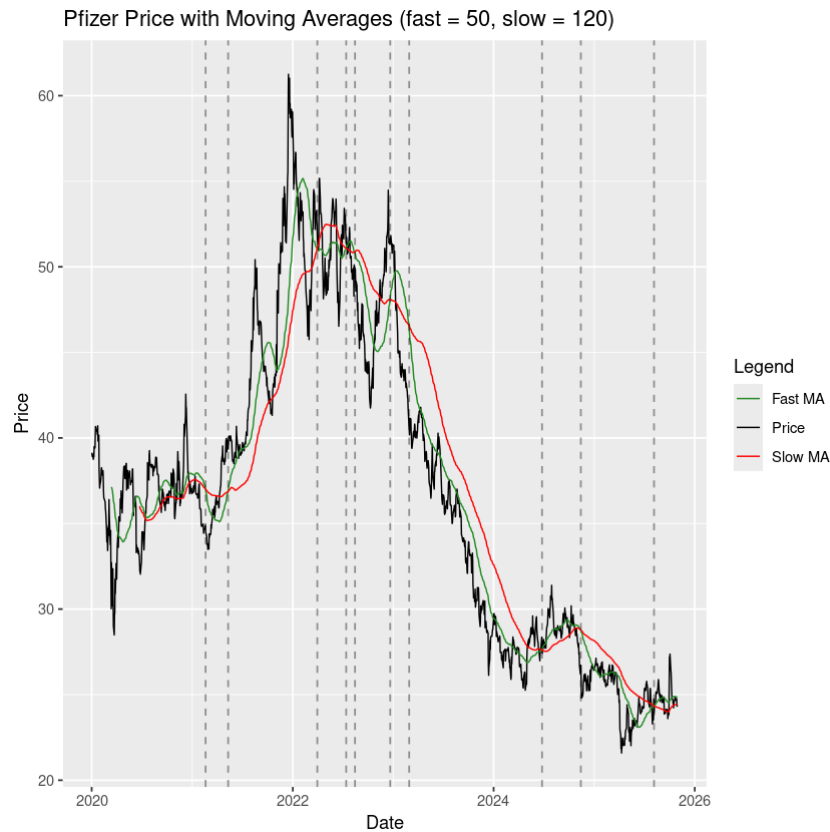


Figure 4: Line Plot Pfizer Price with Moving Averages

Part 5.3: Performance Summary for the Best Strategy

```
# 2) Backtest
ma_best_results <- backtest(
  xts_prices = pfe_price_xts,
  xts_signals = pfe_best[, "signal"]
)

# 3) Plot performance summary
PerformanceAnalytics::charts.PerformanceSummary(
  ma_best_results,
  main = paste0(
    "Market vs Strategy (Best MA Crossover: fast=",
    best_fast, ", slow=", best_slow, ")"
  )
)
```


)
)

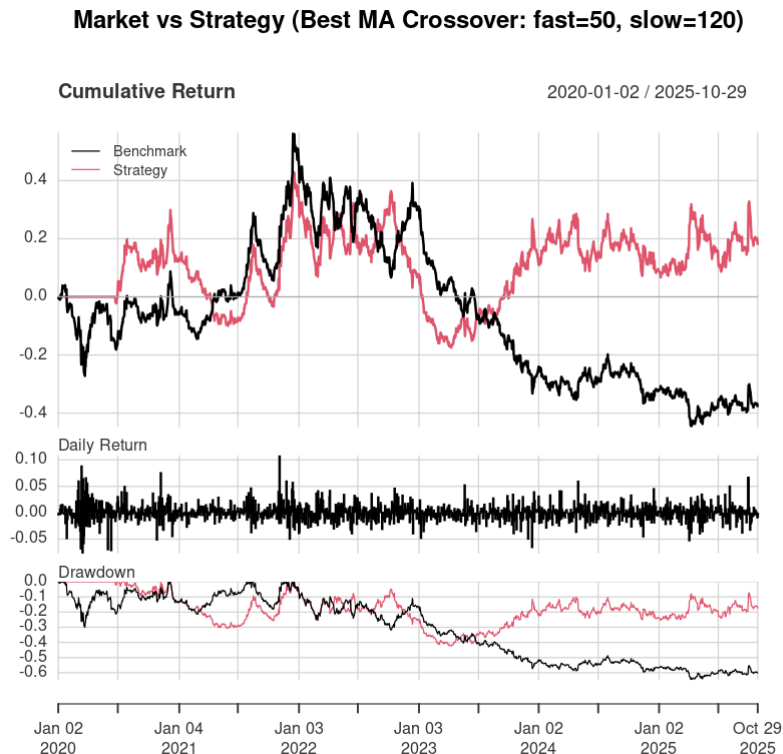


Figure 5: Best Performing Strategy of Moving Average Crossover Strategy

Part 6: Bollinger Bands Breakout Strategy

In this section, I have mobilised the usage of AI, specifically ChatGPT (OpenAI, 2025) to aid me with the production of this strategy.

```

bollinger_breakout_signals_xts <- function(price_xts, window = 20, width_k = 2) {
  # rolling centre + volatility
  bb_mid <- zoo::rollapplyr(price_xts, width = window, FUN = mean, fill = NA, na.rm = TRUE)
  bb_sd <- zoo::rollapplyr(price_xts, width = window, FUN = sd, fill = NA, na.rm = TRUE)
}

```

```

bb_upper <- bb_mid + width_k * bb_sd
bb_lower <- bb_mid - width_k * bb_sd

# breakout only when price CROSSES a band (not just remains outside)
crossed_up   <- price_xts > bb_upper & stats::lag(price_xts) <= stats::lag(bb_upper)
crossed_down <- price_xts < bb_lower & stats::lag(price_xts) >= stats::lag(bb_lower)

sig_event <- ifelse(crossed_up, 1,
                    ifelse(crossed_down, -1, 0))

# carry last non-zero position forward (trend-following breakout convention)
sig_pos <- zoo::na.locf(sig_event, na.rm = FALSE)
sig_pos[is.na(sig_pos)] <- 0

# lag signal to trade next day (avoid look-ahead bias)
sig_pos <- stats::lag(sig_pos, 1)

out <- cbind(bb_mid, bb_upper, bb_lower, sig_pos)
colnames(out) <- c("bb_mid", "bb_upper", "bb_lower", "signal")
return(out)
}

```

Now we transpose and adapt this AI generated Bollinger Bands onto the 2020+ Pfizer dataset and run the existing backtest strategy using my custom function.

```

# Generate Bollinger breakout signals for Pfizer
pfe_bb_xts <- bollinger_breakout_signals_xts(
  price_xts = pfe_price_xts,
  window    = 20,
  width_k   = 2
)

bb_results <- backtest(
  xts_prices = pfe_price_xts,
  xts_signals = pfe_bb_xts[, "signal"]
)

charts.PerformanceSummary(
  bb_results,
  main = "Market vs Strategy (Bollinger Breakout)"
)

```

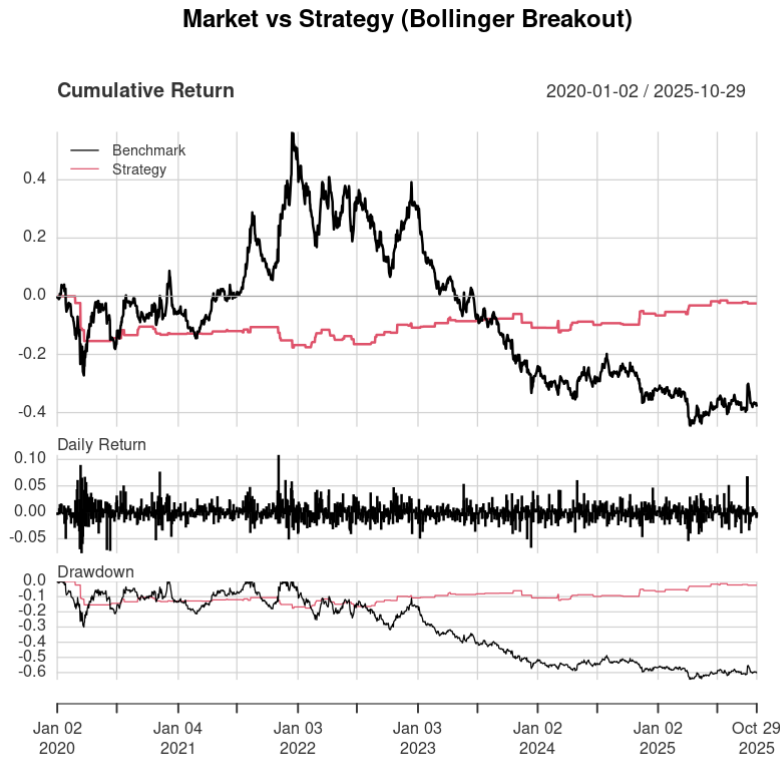


Figure 6: Bollinger Bands Breakout Strategy

Concluding Analysis

The *autocorrelation function (ACF)* (Figure 1 & 2) shows that both daily and weekly returns for Pfizer behave largely like white noise. All bars beyond lag zero fall within the 95% confidence bands, indicating no statistically significant serial dependence in price movements. Weekly mean returns exhibit slightly more fluctuations, but again no autocorrelation exceeds the 0.2 magnitude threshold required to classify it as moderate or statistically significant. This suggests Pfizer's return series is weakly autocorrelated, supporting the weak-form market efficiency hypothesis where simple trend prediction holds limited value.

The *Moving Average Crossover Strategy* (Figure 3) displays consistently weak performance. It falls steeply to approximately -0.45 by early 2020 and remains deeply negative, fluctuating between -0.30 and -0.50 for most of the period. Critically, during the strong bull market phase of 2021-2022 when the benchmark peaked above +0.40, the strategy failed to generate positive

returns. This stark lag and failure to capture trends highlights the limitations of simple trend-following rules in this asset, resulting in a model that failed to add value relative to simply holding the stock.

The *Bollinger Breakout Strategy* (Figure 6) relies on the standard calculation for its bands: a 20-period SMA and 2 standard deviations. Its core lies in the Breakout signal approach, which assumes a cross outside the bands indicates momentum continuation rather than a reversal. While the benchmark is highly volatile—rising nearly 0.5 by early 2022 before collapsing to around -0.4 by 2025, the strategy remains tightly range-bound between -0.1 and -0.15, ending the period near -0.05. This indicates that the strategy, by only entering on a confirmed momentum breakout, fails to capture rapid upside but provides strong downside protection during the major market decline from 2022 onward.

Overall, the strategy delivers superior risk control at the cost of limited return potential, effectively acting as a defensive capital preservation mechanism.